UDP Server Experiment

1. UDP Server Overview

UDP (User Datagram Protocol) is a **connectionless, unreliable but efficient** transport layer protocol. Unlike TCP, UDP does not establish connections, does not guarantee delivery, and does not guarantee ordering, but it has **low overhead and low latency**, making it very suitable for scenarios with high real-time requirements where occasional packet loss is acceptable.

A UDP server does "passive listening" — it binds a port and waits silently, replying only when data is received. Because UDP is connectionless, a "client" is just a source address (IP + port); the server can receive data sent from any source, naturally supporting one-to-many communication.

This experiment supports multiple clients — by recording source addresses, it implements an "whoever comes is a client" mode, with new `clients` and `help` commands added.

Typical application scenarios:

Application Type	Example
DNS server	Receives domain queries and returns IPs
Log collection	Multiple devices send logs to the same port
Command center	Multiple clients send commands to the ESP32

2. Core Concepts

2.1 Server vs Client

	UDP Client	UDP Server
Active handshake	Yes (`HELLO`)	No (silent)
Heartbeat	Every 30s	None
Receive mode	Non-blocking polling	Blocking wait
Client management	Single target	Multiple clients (up to 8 address records)
Extra commands	`ping`, `status`, `led`, `who`	All of the above + `clients`, `help`

2.2 Configuration Parameters

Parameter	Description	Value Used in This Experiment
ESP32_PORT	ESP32 listen port	`8080`
Client limit	Max recorded addresses	8

2.3 Command Table

Complete command set:

Command	Reply	Description
`ping`	`pong`	Connectivity test
`status`	`ESP32 up Xs / free heap: X B`	Running status
`led on`	`LED ON (OK)`	Turn on GPIO48
`led off`	`LED OFF (OK)`	Turn off GPIO48
`who`	`ESP32-S3 / UID: xxx`	Device unique ID
`clients`	List of recorded clients	View historically online devices
`help`	Command list	Help menu
Other text	`echo: <original>`	Echo if unmatched

---

3. Hardware Dependencies

Same as the UDP client. The PC network assistant selects UDP mode; the **target port** is `8080` (the ESP32 bind port).

---

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  |- wifi_connect(): connect to WiFi (STA mode)
  |- udp_bind(): create UDP socket -> bind(8080), blocking by default
  |
  |- while True:
       |- WiFi disconnect watchdog
       |- udp_recv(blocking=True): block and wait for data
       |- _track_client(): record source address
       |- handle_command(): command dispatch
```

---

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> UDP_Server -> UDP_Server.py`

5.1 Creating and Binding the Socket

`SOCK_DGRAM` = UDP; `bind("0.0.0.0", port)` listens on all network interfaces. The server is in blocking mode by default — `recvfrom` keeps waiting for data, saving CPU.

```python
def udp_bind(local_port):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # UDP socket
    s.bind(("0.0.0.0", local_port))                       # bind port
    return s
```

5.2 Send/Receive Functions

`sendto` requires specifying the target address each time; `recvfrom` returns the data + source address for easy replying. The server receives in blocking mode by default; non-blocking is achieved with `settimeout(0.3)`.

```python
def udp_send(sock, ip, port, msg):
    payload = msg.encode("utf-8") if isinstance(msg, str) else msg  # encode
    sock.sendto(payload, (ip, port))                                 # send
    print(f"[TX] -> {ip}:{port}  |  {msg}")

def udp_recv(sock, bufsize=256, blocking=True):
    if not blocking:
        sock.settimeout(0.3)                                    # non-blocking
0.3s
        try:
            data, addr = sock.recvfrom(bufsize)                 # receive
            return data, addr
        except OSError:
            return None, None                                    # timeout
    data, addr = sock.recvfrom(bufsize)                          # blocking
recv
    return data, addr

def fmt(data):
    try:
        return data.decode("utf-8")                              # decode
```

```
    except UnicodeError:
        return str(data)                                    # fallback
```

5.3 Multi-Client Tracking

A `{ "ip:port": last_seen_time }` dictionary records clients, with a maximum of 8; when the limit is exceeded, the oldest one is evicted.

```
_clients = {}                                  # { "ip:port": last_seen_time }

def _track_client(ip, port):
    key = f"{ip}:{port}"
    if key not in _clients and len(_clients) >= 8:
        oldest = min(_clients, key=lambda k: _clients[k])
        del _clients[oldest]                   # cap at 8
    _clients[key] = time.time()
```

5.4 Command Dispatch

Compared to the client, the server adds `clients` and `help` commands; `handle_command` dispatches based on the command string.

```
def handle_command(sock, addr, cmd):
    cmd_lower = cmd.strip().lower()

    if cmd_lower == "ping":
        udp_send(sock, *addr, "pong")                          # ping-pong

    elif cmd_lower == "status":
        import gc; gc.collect()
        mem_free = gc.mem_free()
        reply = f"ESP32 up {time.time():.0f}s | free heap: {mem_free} B"
        udp_send(sock, *addr, reply)                           # status

    elif cmd_lower == "led on":
        from machine import Pin
        Pin(48, Pin.OUT, value=1)                               # LED on
        udp_send(sock, *addr, "LED ON (OK)")

    elif cmd_lower == "led off":
        from machine import Pin
        Pin(48, Pin.OUT, value=0)                               # LED off
        udp_send(sock, *addr, "LED OFF (OK)")

    elif cmd_lower == "who":
        import machine, ubinascii
        uid = ubinascii.hexlify(machine.unique_id()).decode()  # UID
        udp_send(sock, *addr, f"ESP32-S3 UID: {uid}")

    elif cmd_lower == "clients":                                # tracked
        lines = [f"{k} | last seen {time.time()-v:.0f}s ago"
                 for k, v in _clients.items()]
        udp_send(sock, *addr, "\n".join(lines) if lines else "No clients.")

    elif cmd_lower == "help":                                   # help menu
        udp_send(sock, *addr, "Commands: ping|status|led|who|clients|help")
```

```
    else:
        udp_send(sock, *addr, f"echo: {cmd}")                    # echo
```

5.5 Main Entry

`wifi_connect` connects to WiFi -> `udp_bind` binds the port -> main loop: blocking `udp_recv` waits for data -> `fmt` decodes -> `_track_client` records -> `handle_command` dispatches the reply.

```
def main():
    wlan = wifi_connect(WIFI_SSID, WIFI_PASSWORD)  # connect to WiFi
    esp_ip = wlan.ifconfig()[0]

    sock = udp_bind(ESP32_PORT)                     # bind port
    print(f"[SOCK] UDP bound to 0.0.0.0:{ESP32_PORT}")

    while True:
        if not wlan.isconnected():                  # WiFi watchdog
            wlan = wifi_connect(WIFI_SSID, WIFI_PASSWORD)
        data, addr = udp_recv(sock, blocking=True)  # blocking recv
        if data:
            msg = fmt(data)
            _track_client(addr[0], addr[1])          # track client
            handle_command(sock, addr, msg)          # dispatch
```

---

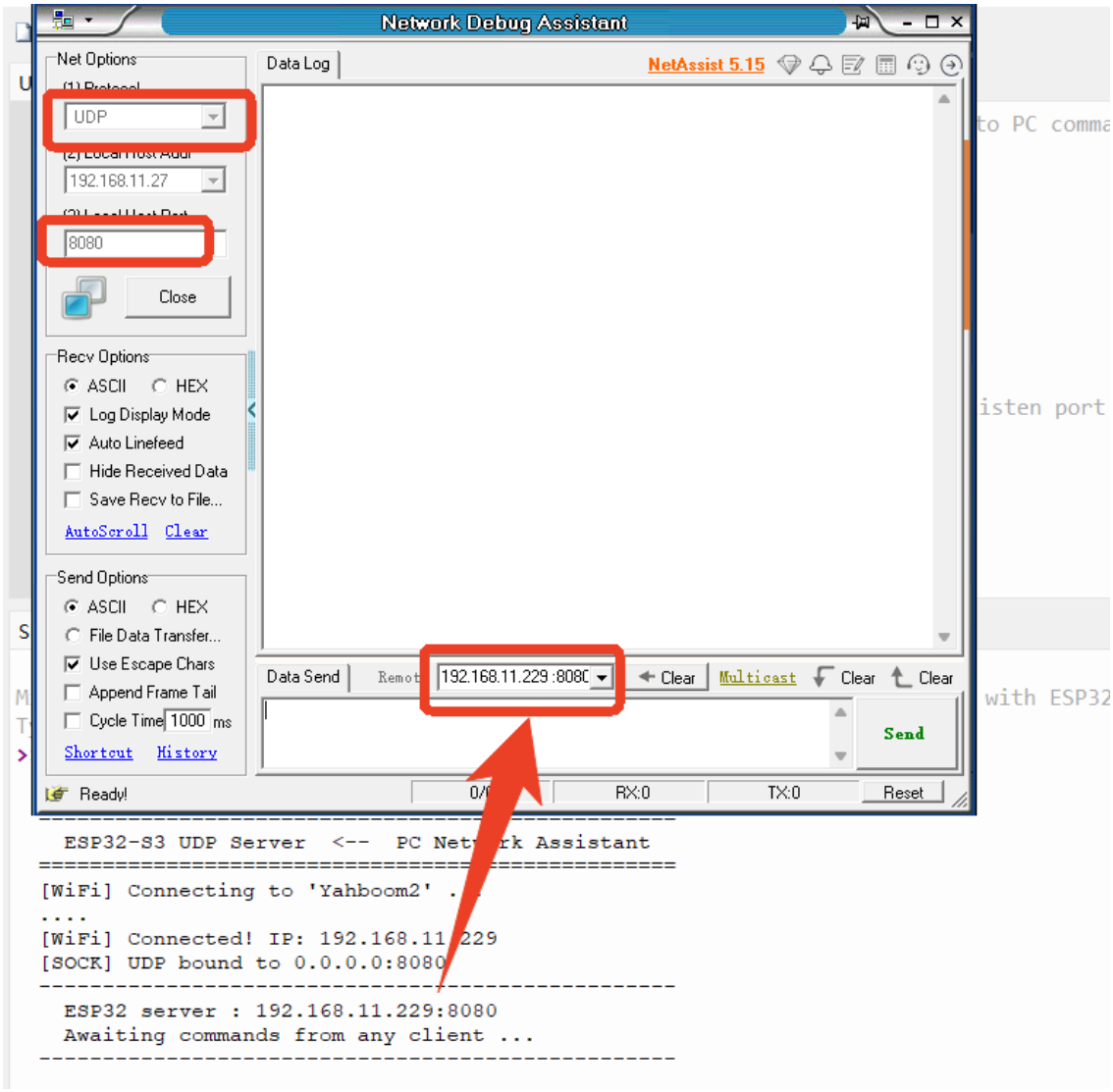
6. Operating Procedures

6.1 Environment Preparation

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3.Thonny IDE`.

1. PC network debugging assistant 

UDP mode; the target IP is the ESP32's IP, and the target port is `8080`.



2. Configure `WIFI_SSID` / `WIFI_PASS` / `PC_PORT`

```
# ==================== 用户配置 / User Config ====================
WIFI_SSID     = "Yahboom2"          # WiFi名称 / WiFi SSID
WIFI_PASSWORD = "yahboom890729"     # WiFi密码 / WiFi password

ESP32_PORT    = 8080              # ESP32-本地监听端口 / ESP32 local listen port
# ================================================================
```

6.2 Flashing and Running

Run `UDP_Server.py`; after the ESP32 binds the port, it listens silently, and the PC can send commands and receive replies.

6.3 Serial Output Example

```
==================================================
  ESP32-S3 UDP Server  <--  PC Network Assistant
==================================================
[WiFi] Connecting to 'Yahboom2' ...
...
[WiFi] Connected! IP: 192.168.11.229
[SOCK] UDP bound to 0.0.0.0:8080
--------------------------------------------------
  ESP32 server : 192.168.11.229:8080
  Awaiting commands from any client ...
--------------------------------------------------
```

```
[RX] <- 192.168.11.27:8080  |  status
[TX] -> 192.168.11.27:8080  |  ESP32 up 839319800s | free heap: 8315632 B
[RX] <- 192.168.11.27:8080  |  ping
[TX] -> 192.168.11.27:8080  |  pong
[RX] <- 192.168.11.27:8080  |  clients
[TX] -> 192.168.11.27:8080  |  Tracked clients (1):
192.168.11.27:8080 | last seen 0s ago
[RX] <- 192.168.11.27:8080  |  help
[TX] -> 192.168.11.27:8080  |  Commands: ping | status | led on | led off | who |
clients | help
[RX] <- 192.168.11.27:8080  |  wh0
[TX] -> 192.168.11.27:8080  |  echo: wh0
```

6.4 Verification Methods

Operation	Expected Result
Two PCs send `ping` at the same time	Both receive `pong`
Send `clients`	Lists the recorded client addresses
Send `help`	Returns the command list

---

7. Common Problems

Symptom	Cause	Solution
No reply after sending a command	Target port not set to `8080`	The PC network assistant's target port should be ESP32_PORT
Multiple clients but only one recorded	Other PCs have different target ports	UDP is connectionless; the source port depends on the client's local port